

C++ - Module 08

Templated containers, iterators, algorithms

Özet: Bu doküman, C++ modüllerinin 08. Modülüne ait alıştırmaları içermektedir.

Versiyon: 10.1

İçindekiler

- I. Giriş — 2
 - II. Genel kurallar — 3
 - III. Modüle özgü kurallar — 6
 - IV. AI Talimatları — 7
 - V. Exercise 00: Easy find — 9
 - VI. Exercise 01: Span — 10
 - VII. Exercise 02: Mutated abomination — 12
 - VIII. Teslim ve akran değerlendirmesi — 14
-

Chapter I — Giriş

C++, Bjarne Stroustrup tarafından C programlama dilinin bir uzantısı olarak yaratılmış, genel amaçlı bir programlama dilidir; ya da "Sınıflarla C" olarak da bilinir (kaynak: Wikipedia).

Bu modüllerin amacı sizi **Nesne Yönelimli Programlamaya** (Object-Oriented Programming) tanıtmaktır.

Bu, C++ yolculuğunuzun başlangıç noktası olacak. OOP öğrenmek için pek çok dil tavsiye edilir. Biz C++'ı seçtik, çünkü eski dostunuz C'den türemiştir.

Bu karmaşık bir dil olduğundan ve işleri basit tutmak adına, kodunuz C++98 standardına uygun olacaktır.

Modern C++'ın pek çok açıdan oldukça farklı olduğunun farkındayız. Dolayısıyla yetkin bir C++ geliştiricisi olmak istiyorsanız, 42 Common Core'dan sonra daha ileri gitmek size kalmış!

Chapter II — Genel kurallar

Derleme

- Kodunuzu `C++` ve `-Wall -Wextra -Werror` flag'leriyle derleyin.
- Kodunuz `-std=c++98` flag'i eklendiğinde de derlenebilmelidir.

Biçimlendirme ve adlandırma kuralları

- Alıştırma dizinleri şu şekilde adlandırılacak: `ex00`, `ex01`, ..., `exn`
- Dosyalarınızı, sınıflarınızı, fonksiyonlarınızı, üye fonksiyonlarınızı ve attribute'lerinizi yönergelerde istenildiği gibi adlandırın.
- Sınıf isimlerini **UpperCamelCase** formatında yazın. Sınıf kodunu içeren dosyalar her zaman sınıf ismine göre adlandırılacaktır. Örneğin: `ClassName.hpp/ClassName.h`, `ClassName.cpp`, ya da `ClassName.hpp`. Dolayısıyla, "BrickWall" (tuğla duvar) adında bir sınıfın tanımını içeren bir header dosyanız varsa, ismi `BrickWall.hpp` olacaktır.
- Aksi belirtilmedikçe, her çıktı mesajı bir newline karakteri ile bitmeli ve standart çıktıya (standard output) gösterilmelidir.
- Norminette'e elveda! C++ modüllerinde herhangi bir kodlama stili zorunlu kılınmamıştır. Kendi favori stilinizi takip edebilirsiniz. Ancak unutmayın ki ekran değerlendiricilerinizin anlayamadığı kod, not veremeyecekleri koddur. Temiz ve okunabilir kod yazmak için elinizden geleni yapın.

İzin verilenler / Yasaklananlar

Artık C ile kod yazmıyorsunuz. C++ zamanı! Bu nedenle:

- Standart kütüphaneden hemen hemen her şeyi kullanmanıza izin verilir. Dolayısıyla, alıştığınız şeylere bağlı kalmak yerine, alışkın olduğunuz C fonksiyonlarının C++'a özgü versiyonlarını mümkün olduğunca kullanmak akıllıca olacaktır.
- Ancak, başka herhangi bir harici kütüphane kullanamazsınız. Bu, C++11'in (ve türevlerinin) ve Boost kütüphanelerinin yasak olduğu anlamına gelir. Şu fonksiyonlar da yasaktır: `*printf()`, `*alloc()` ve `free()`. Bunları kullanırsanız notunuz 0 olur, hepsi bu kadar.
- Aksi açıkça belirtilmedikçe, `using namespace <ns_name>` ve `friend` anahtar kelimeleri yasaktır. Aksi takdirde notunuz -42 olur.
- STL'i yalnızca Module 08 ve 09'da kullanmanıza izin verilir. Bu, o zamana kadar Container'ların (vector/list/map vb.) ve Algorithm'lerin (`<algorithm>` header'ını içermeyi gerektiren her şey) yasak olduğu anlamına gelir. Aksi takdirde notunuz -42 olur.

Birkaç tasarım gereksinimi

- Bellek sızıntısı (memory leakage) C++'ta da yaşanır. `new` anahtar kelimesini kullanarak bellek ayırdığınızda, bellek sızıntılarından (memory leaks) kaçınmalısınız.

- Module 02'den Module 09'a kadar, sınıflarınız aksi açıkça belirtilmedikçe **Orthodox Canonical Form**da tasarlanmalıdır.
- Bir header dosyasına konulan herhangi bir fonksiyon implementasyonu (function template'ler hariç) alıştırma için 0 anlamına gelir.
- Header'larınızın her birini diğerlerinden bağımsız olarak kullanabilmelisiniz. Dolayısıyla, ihtiyaç duydukları tüm bağımlılıkları include etmelidirler. Ancak, include guard'lar ekleyerek çift include (double inclusion) sorunundan kaçınmalısınız. Aksi takdirde notunuz 0 olur.

Beni oku (Read me)

- İhtiyaç duyarsanız bazı ek dosyalar ekleyebilirsiniz (örn. kodunuzu bölmek için). Bu ödevler bir program tarafından kontrol edilmediğinden, zorunlu dosyaları teslim ettiğiniz sürece bunu yapmaktan çekinmeyin.
- Bazen, bir alıştırmanın yönergeleri kısa görünür ama örnekler, talimatlarda açıkça yazılmamış gereksinimleri gösterebilir.
- Başlamadan önce her modülü baştan sona okuyun! Gerçekten yapın.
- Odin aşkına, Thor aşkına! Beyninizi kullanın!!!

Uyarı: C++ projeleri için Makefile konusunda, C'deki ile aynı kurallar geçerlidir (Norm'un Makefile bölümüne bakınız).

İpucu: Pek çok sınıf implement etmeniz gerekecek. Bu sıkıcı görünebilir, tabii favori metin editörünüzü script'leyebiliyorsanız durum değişir.

Bilgi: Alıştırmaları tamamlamak için size belirli bir özgürlük tanınıyor. Ancak, zorunlu kurallara uyun ve tembellik etmeyin. Aksi takdirde pek çok faydalı bilgiyi kaçırmış olursunuz! Teorik kavramlar hakkında okumaktan çekinmeyin.

Chapter III — Modüle özgü kurallar

Bu modülde, alıştırmaların standart Container'lar KULLANILMADAN ve standart Algorithm'ler KULLANILMADAN da çözülebileceğini fark edeceksiniz.

Ancak, **bunları kullanmak tam olarak bu Modülün amacıdır.**

STL'i kullanmak ZORUNDASINIZ — özellikle **Container'ları** (vector/list/map ve benzerleri) ve **Algorithm'leri** (`<algorithm>` header'ında tanımlı olanları) — uygun oldukları her yerde. Dahası, onları mümkün olduğunca çok kullanmalısınız. Bu nedenle, uygun olan her yerde kullanmak için elinizden gelenin en iyisini yapın.

Kullanmazsanız, kodunuz beklendiği gibi çalışsa bile çok kötü bir not alırsınız. Lütfen tembel olmayın.

Template'lerinizi her zamanki gibi header dosyalarında tanımlayabilirsiniz. Ya da isterseniz, template bildirimlerinizi (declaration) header dosyalarına,

implementasyonlarını ise .tpp dosyalarına yazabilirsiniz. Her halükarda, header dosyaları zorunlu, .tpp dosyaları ise isteğe bağlıdır.

Chapter IV — AI Talimatları

Bağlam (Context)

Bu proje, 42 eğitiminizin temel yapı taşlarını keşfetmenize yardımcı olmak için tasarlanmıştır.

Temel bilgi ve becerileri doğru şekilde yerleştirmek için, AI araçlarını ve desteğini kullanma konusunda düşünceli bir yaklaşım benimsemek esastır.

Gerçek temel öğrenme, zorluk, tekrar ve akranlarla öğrenme alışverişi yoluyla gerçek bir entelektüel çaba gerektirir.

AI hakkındaki tutumumuza dair daha kapsamlı bir genel bakış için — bir öğrenme aracı olarak, 42 eğitiminin bir parçası olarak ve iş piyasasında bir beklenti olarak — lütfen intranetteki ilgili SSS'ye (FAQ) bakınız.

Ana mesaj

- Kestirmelere başvurmadan sağlam temeller inşa edin.
- Tech & power skills'inizi gerçekten geliştirin.
- Gerçek akran öğrenmeyi deneyimleyin, nasıl öğreneceğinizi ve yeni problemleri nasıl çözeceğinizi öğrenmeye başlayın.
- Öğrenme yolculuğu sonuçtan daha önemlidir.
- AI ile ilişkili riskler hakkında bilgi edinin ve yaygın tuzaklardan kaçınmak için etkili kontrol pratikleri ve karşı önlemler geliştirin.

Öğrenci kuralları

- Görevlerinize, özellikle AI'ya başvurmadan önce, akıl yürütme uygulamalısınız.
- AI'dan doğrudan cevap istememelisiniz.
- 42'nin AI konusundaki genel yaklaşımını öğrenmelisiniz.

Aşama çıktıları

Bu temel aşama içinde şu çıktıları elde edeceksiniz:

- Düzgün tech ve kodlama temelleri kazanmak.
- Bu aşamada AI'nin neden ve nasıl tehlikeli olabileceğini bilmek.

Yorumlar ve örnek

- Evet, AI'nin var olduğunu biliyoruz — ve evet, projelerinizi çözebilir. Ama siz burada

öğrenmek için varsınız, AI'nin öğrendiğini kanıtlamak için değil. AI'nin verilen problemi çözebildiğini göstermek için sadece zamanınızı (veya bizimkini) boşa harcamayın.

- 42'de öğrenmek, cevabı bilmekle ilgili değildir — bir cevap bulma yeteneği geliştirmekle ilgilidir. AI size cevabı doğrudan verir, ama bu sizin kendi akıl yürütmenizi inşa etmenizi engeller. Akıl yürütme zaman, çaba gerektirir ve başarısızlığı da içerir. Başarıya giden yol kolay olması gerektiği şekilde tasarlanmamıştır.
- Sınavlar sırasında AI'nin mevcut olmadığını unutmayın — internet yok, akıllı telefon yok, vb. Öğrenme sürecinizde AI'ye fazlasıyla güvenip güvenmediğinizi hızlıca fark edeceksiniz.
- Akran öğrenmesi sizi farklı fikirlere ve yaklaşımlara maruz bırakır, kişilerarası becerilerinizi ve farklı şekillerde düşünme yeteneğinizi geliştirir. Bu, sadece bir botla sohbet etmekten çok daha değerlidir. O yüzden çekinmeyin — konuşun, sorular sorun ve birlikte öğrenin!
- Evet, AI müfredatın bir parçası olacak — hem bir öğrenme aracı hem de kendi başına bir konu olarak. Hatta kendi AI yazılımınızı inşa etme şansınız bile olacak. Gececeğiniz crescendo yaklaşımımız hakkında daha fazla bilgi edinmek için intranette mevcut dokümantasyona bakın.

✓ **İyi pratik:** Yeni bir kavramda takılı kaldım. Yakınımdaki birine bu konuya nasıl yaklaştığını soruyorum. 10 dakika konuşuyoruz — ve birden anlıyorum. Kavradım.

✗ **Kötü pratik:** Gizlice AI kullanıyorum, doğru görünen bir kod kopyalıyorum. Akran değerlendirmesi sırasında hiçbir şeyi açıklayamıyorum. Başarısız oluyorum. Sınavda — AI yok — yine takılıyorum. Başarısız oluyorum.

Chapter V — Exercise 00: Easy find

Exercise	00
	Easy find
Directory	<code>ex00/</code>
Teslim edilecek dosyalar	<code>Makefile</code> , <code>main.cpp</code> , <code>easyfind.{h, hpp}</code>
Opsiyonel dosya	<code>easyfind.tpp</code>
Yasaklı	Yok

İlk kolay alıştırma, doğru bir başlangıç yapmanın yoludur.

T tipini kabul eden bir `easyfind` function template'i yazın. İki parametre alır: birincisi T tipinde, ikincisi ise bir integer'dır.

T'nin **integer'lardan oluşan bir container** olduğunu varsayarak, bu fonksiyonun birinci parametrede ikinci parametrenin ilk geçtiği yeri bulması gerekir.

Eğer hiçbir eşleşme bulunamazsa, kendi seçeceğiniz bir exception fırlatabilir ya da bir hata değeri döndürebilirsiniz. İlham almak istiyorsanız, standart container'ların nasıl davrandığını inceleyin.

Elbette, her şeyin beklendiği gibi çalıştığından emin olmak için kendi testlerinizi implement edip teslim edin.

Bilgi: Associative container'ları ele almanız gerekmiyor.

Chapter VI — Exercise 01: Span

Exercise	01
	Span
Directory	<code>ex01/</code>
Teslim edilecek dosyalar	<code>Makefile</code> , <code>main.cpp</code> , <code>Span.{h, hpp}</code> , <code>Span.cpp</code>
Yasaklı	Yok

Maksimum N adet integer saklayabilen bir `Span` sınıfı geliştirin. N, unsigned int türünde bir değişkendir ve constructor'a geçilecek tek parametredir.

Bu sınıfın, Span'a tek bir sayı eklemek için `addNumber()` adında bir üye fonksiyonu olacaktır. Span'ı doldurmak için bu fonksiyon kullanılacaktır. Zaten N eleman depolanmışken yeni bir eleman ekleme girişimi bir exception fırlatmalıdır.

Ardından iki üye fonksiyon implement edin: `shortestSpan()` ve `longestSpan()`

Bunlar, depolanan tüm sayılar arasındaki en kısa span'i ya da en uzun span'i (veya isterseniz mesafeyi) bulacak ve döndürecektir. Eğer hiç sayı depolanmamışsa ya da yalnızca bir tane varsa, span bulunamaz. Dolayısıyla, bir exception fırlatın.

Elbette, kendi testlerinizi yazacaksınız ve bunlar aşağıdakilerden çok daha kapsamlı olacak. `Span`'ınızı en az 10.000 sayıyla test edin. Daha fazlası daha da iyi olur.

Örnek

Aşağıdaki kodu çalıştırmak:

cpp

```
int main()
{
    Span sp = Span(5);

    sp.addNumber(6);
    sp.addNumber(3);
    sp.addNumber(17);
    sp.addNumber(9);
    sp.addNumber(11);

    std::cout << sp.shortestSpan() << std::endl;
    std::cout << sp.longestSpan() << std::endl;

    return 0;
}
```

Şu çıktıyı vermeli:

```
$> ./ex01
2
14
$>
```

Son olarak, `Span`'ınızı **bir iterator aralığı (range of iterators)** kullanarak doldurmak harika olurdu. `addNumber()`'ı binlerce kez çağırmak oldukça can sıkıcı. `Span`'ınıza tek bir çağrıda birden fazla sayı ekleyecek bir üye fonksiyon implement edin.

İpucu: Eğer fikriniz yoksa, Container'ları inceleyin. Bazı üye fonksiyonlar, container'a bir dizi eleman eklemek için bir iterator aralığı alır.

Chapter VII — Exercise 02: Mutated abomination

Exercise

02

Mutated abomination

Directory

ex02/

Teslim edilecek dosyalar

Makefile, main.cpp, MutantStack.{h, hpp}

Opsiyonel dosya

MutantStack.hpp

Yasaklı

Yok

Artık daha ciddi şeylere geçme zamanı. Tuhaf bir şey geliştirelim.

`std::stack` container'ı çok güzel. Ne yazık ki, iterate edilemeyen (NOT iterable) tek STL Container'larından biridir. Bu çok kötü.

Ama neden bunu kabul edelim? Özellikle de eksik özellikleri yaratmak için orijinal stack'i parçalama özgürlüğünü alabilecekken.

Bu adaletsizliği gidermek için, `std::stack` container'ını iterate edilebilir hale getirmeniz gerekiyor.

Bir `MutantStack` sınıfı yazın. Bu sınıf `std::stack` kullanılarak implement edilecektir.

`std::stack`'in tüm üye fonksiyonlarını sunacak, artı ek bir özellik: **iterator'lar**.

Elbette, her şeyin beklendiği gibi çalıştığından emin olmak için kendi testlerinizi yazıp teslim edeceksiniz.

Test örneği

```
cpp
```



```

int main()
{
    MutantStack<int> mstack;

    mstack.push(5);
    mstack.push(17);

    std::cout << mstack.top() << std::endl;

    mstack.pop();

    std::cout << mstack.size() << std::endl;

    mstack.push(3);
    mstack.push(5);
    mstack.push(737);
    //[...]
    mstack.push(0);

    MutantStack<int>::iterator it = mstack.begin();
    MutantStack<int>::iterator ite = mstack.end();

    ++it;
    --ite;
    while (it != ite)
    {
        std::cout << *it << std::endl;
        ++it;
    }
    std::stack<int> s(mstack);
    return 0;
}

```

Bunu ilk kez `MutantStack`'inizle, ikinci kez ise `MutantStack`'i örneğin `std::list` ile değiştirerek çalıştırırsanız, iki çıktı da aynı olmalıdır. Elbette, başka bir container test ederken, kodu ilgili üye fonksiyonlarla güncelleyin (`push()` yerine `push_back()` kullanılabilir).

Chapter VIII — Teslim ve akran değerlendirmesi

Ödevinizi her zamanki gibi Git repository'nize teslim edin. Savunma (defense) sırasında yalnızca repository'niz içindeki çalışma değerlendirilecektir. Klasör ve dosya isimlerinizin doğru olduğundan emin olmak için kontrol etmekten çekinmeyin.

Değerlendirme sırasında, projede kısa bir **modifikasyon** ara sıra talep edilebilir. Bu, küçük bir davranış değişikliği, yazılacak veya yeniden yazılacak birkaç satır kod, ya da eklemesi kolay bir özellik içerebilir.

Bu adım her proje için geçerli olmayabilir, ancak değerlendirme yönergelerinde belirtilmişse buna hazırlıklı olmalısınız.

Bu adım, projenin belirli bir bölümünü gerçekten anlayıp anlamadığınızı doğrulamak içindir. Modifikasyon, seçtiğiniz herhangi bir geliştirme ortamında (örn. kendi kullandığınız kurulum) yapılabilir ve değerlendirmenin bir parçası olarak belirli bir süre tanımlanmadıkça birkaç dakika içinde gerçekleştirilebilir olmalıdır.

Örneğin, bir fonksiyonda veya script'te küçük bir güncelleme yapmanız, bir görüntüyü (display) değiştirmeniz, ya da yeni bilgi saklamak için bir veri yapısını ayarlamanız vb. istenebilir.

Detaylar (kapsam, hedef vb.) değerlendirme yönergelerinde belirtilecektir ve aynı proje için bir değerlendirmeden diğerine farklılık gösterebilir.